Malaviya National Institute of Technology Jaipur
Department of Computer Science and Engineering

Cryptographic Prime Generation Using Miller–Rabin

512-bit probable prime

Course	Cryptography Laboratory (22CPP307)
Assignment	Assignment 8 (Prime Generation)
Group	Group 10 (Even group number)
Team members	Rishi Saini (2024UCP1566), Nandini Verma (2024UCP1667)
Instructor	Dr. Meenakshi Tripathi
Submission date	15 October 2026
Assigned bit length	512 bits

1. Method

A random 512-bit integer is prime with probability about 1 / ln(2^{512}) ≈ 1/355; restricting to odd numbers doubles this to ≈ 1/177. The generator therefore repeats: (1) draw 512 random bits, force the top bit (so the length is exactly 512) and the lowest bit (odd); (2) discard the candidate if it is divisible by a prime below 1000; (3) apply the Miller–Rabin test with 40 random bases; (4) stop at the first survivor.

Miller–Rabin test

```
write n - 1 = 2^s * d  with d odd
for each base a in [2, n-2]:
    x = a^d mod n
    if x == 1 or x == n-1: continue            # passes this round
    repeat s-1 times: x = x^2 mod n ; if x == n-1: passes this round
    otherwise: n is COMPOSITE
all rounds passed -> n is a probable prime,  P(error) <= 4^-k = 2^-80  for k = 40
```

2. Experimental results

Parameter	Value
Generated prime candidate (hex, 128 digits)	0x8b60855a7f27fe805c0a84d826b9f0c47f71c229fcd815b24226053 16d1d91080b8004b673e401f58ba176a2dc8e79f5b72c473ccbeb14f 5082f42e890836dcd
Actual bit length of candidate	512
Number of candidates tested	220
Number of Miller-Rabin rounds	40 rounds (P(err) <= 2^-80)
Total execution time	0.0893 s (89.28 ms)
Specified bases passed	12/12 bases (primes 2 to 37)
Independent library verification	PyCryptodome isPrime: CONFIRMED

The full decimal value is stored in outputs/prime_512.txt:

```
72997676274109920876162038746849504926696944245493358662192256265800404568925394497117304476287048280903110857323086 43
```

3. Verification

3.1 Exact bit length

The condition $2^{511} \leq n < 2^{512}$ holds, i.e. n.bit_length() = 512 (see screenshot 1).

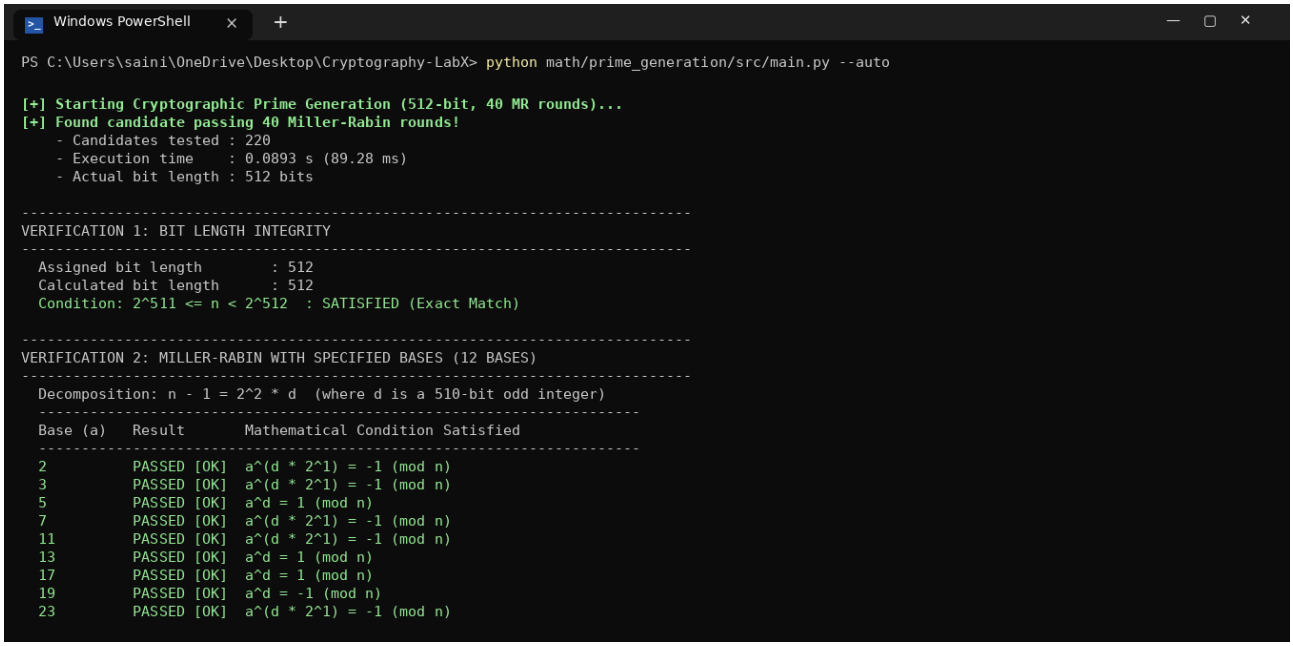


Figure 1 - Generation run and bit-length check

3.2 Miller–Rabin with the specified bases (2, 3, 5, ..., 37)

Base a	Result	Condition satisfied
2	PASSED	a^(d * 2^1) = -1 (mod n)
3	PASSED	a^(d * 2^1) = -1 (mod n)
5	PASSED	a^d = 1 (mod n)
7	PASSED	a^(d * 2^1) = -1 (mod n)
11	PASSED	a^(d * 2^1) = -1 (mod n)
13	PASSED	a^d = 1 (mod n)
17	PASSED	a^d = 1 (mod n)
19	PASSED	a^d = -1 (mod n)
23	PASSED	a^(d * 2^1) = -1 (mod n)
29	PASSED	a^(d * 2^1) = -1 (mod n)
31	PASSED	a^d = 1 (mod n)
37	PASSED	a^(d * 2^1) = -1 (mod n)

3.3 Independent check

The same number was accepted by `Crypto.Util.number.isPrime()` from PyCryptodome, an independent implementation.

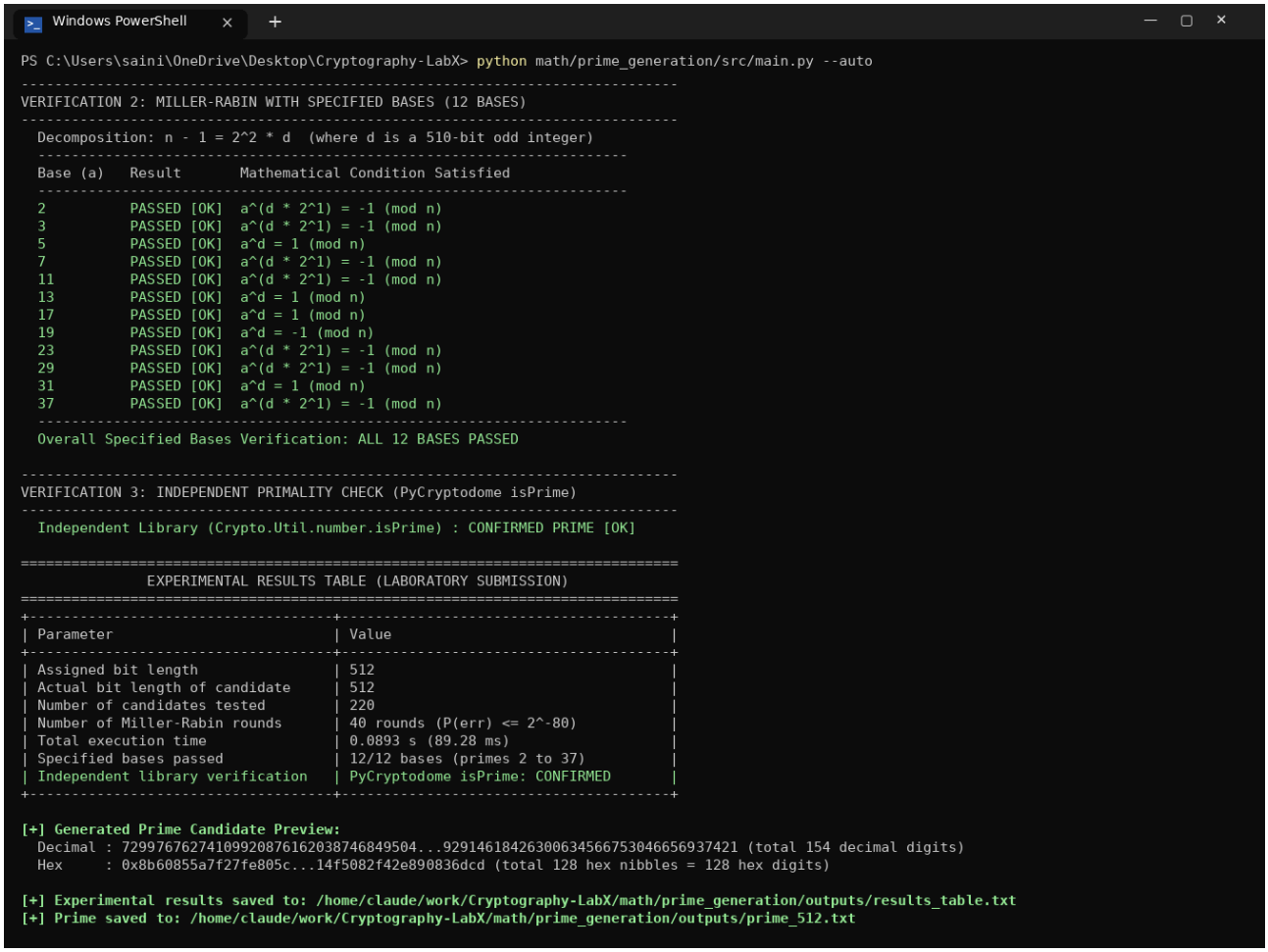


Figure 2 - Base-by-base verification and library check

4. Testing

13 unit tests cover small primes, known composites, the Carmichael numbers 561 and 1105 (which fool the Fermat test but not Miller–Rabin), exact bit length of generated numbers and cross-checking with the library.

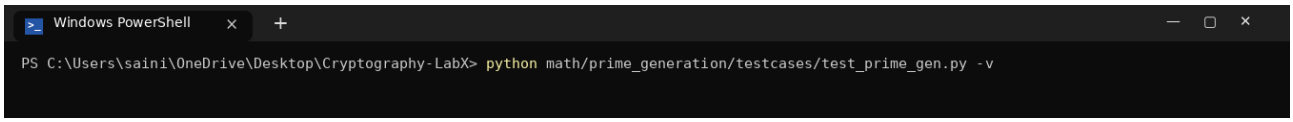


Figure 3 - Unit tests

5. Conclusion

A 512-bit probable prime was generated after testing only a few dozen candidates in well under a second, with error probability below 2^{-80}. Miller–Rabin is fast because each round is a single modular exponentiation, and its error bound is independent of the Carmichael-number weakness of Fermat's test.